

Context-Aware Digital Twin for Polymer-Joint Cobots

Next Steps, Mathematical Background, and Copilot Agent Data-Logging Brief

Current priority	Make the repository identification-ready, collect clean normal-operation data, and compare Model A vs Model B offline before changing VFF-RLS or the context state machine.
Robot / repository	6-DOF igus ReBeL; repository: dungnguyen15213/6DOF-Rebel-Cobot-Control.
Primary output needed now	A dedicated raw telemetry CSV with reliable timestamps, run metadata, joint position, joint velocity, and measured current. Acceleration is a candidate signal, not yet trusted.
Decision gate	Freeze the final current regressor only after held-out-run evidence shows whether reconstructed acceleration materially improves prediction.

Source basis. Project status, sequence, model definitions, logger requirements, and research guardrails are based on the uploaded Context-Aware Digital Twin Roadmap - 25 Sep 2026. The explanatory derivations below expand the mathematics for understanding; they are not quoted equations from the roadmap.

1. What to Do Next

The research bottleneck is not the GUI or robot communication layer. It is the scientific model that predicts normal motor current. The safest sequence is to establish a defensible offline model first, then make it adaptive.

- Prepare the logger. Preserve raw CRI STATUS telemetry. Add a dedicated experiment CSV instead of reusing an arbitrary diagnostics file.
- Run a short dry test. Check timestamp spacing, missing samples, duplicate timestamps, units, sign conventions, and current clipping/saturation.
- Collect normal-operation data. Start with payload = 0, contact_label = NORMAL, no deliberate contact, and no collision. Include positive and negative motion, several safe speeds, single-joint excitation, and representative multi-joint paths.
- Build deterministic preprocessing. Convert to radians and amperes, use measured Δt , filter velocity before differentiating, generate a candidate acceleration signal, and plot all signals for quality control.
- Fit Model A first. Validate on entire held-out runs.
- Fit Model B second. Add acceleration only after Model A works, then compare with exactly the same train/validation split.
- Freeze the regressor. Only then proceed to thermal drift experiments, VFF-RLS, context gating, state logic, calibration, and final validation.

What you should prepare before going back to the robot

Item	Prepare now	Why it matters
Experiment identifiers	experiment_id, trajectory_id, run_id	Allows repeatable train/validation splits and traceability.
Condition metadata	commanded_speed, payload_g, contact_label, grasp_context, cold/warm	Prevents mixing physically different regimes.
Raw telemetry	timestamp, q1..q6, qdot1..qdot6, current1..6	These are the minimum signals for Model A and the residual.
Candidate acceleration	raw logged acceleration + reconstructed acceleration	Lets you test Model B rather than assuming acceleration is good.
Trajectory plan	positive/negative motion; low/medium/higher safe speed; single + multi-joint	Improves excitation and identifiability.
Validation plan	repeat trajectories as separate runs	Prevents leakage from near-identical neighboring samples.

2. Mathematical Background: From Robot Motion to Motor Current

The core idea is simple: predict the current that should be required during normal motion, then study the difference between predicted and measured current. That difference - the residual - carries information about model error, thermal drift, payload changes, and external contact.

$$\tau = M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q) + f(\dot{q}) + \tau_{\text{ext}}$$

For a robot joint, torque is caused by inertia, velocity-dependent dynamics, gravity, friction, and external torque. For an electric motor, torque is approximately related to motor current by

$$\tau_m \approx K_t I$$

Therefore motor current contains information about how hard the joint is working. Because the ReBeL internal controller, transmissions, friction, and exact motor constants are not fully modeled here, the project uses system identification: learn a compact current model directly from telemetry.

$$I_i(k) = \phi_i(k)^T \theta_i + e_i(k)$$

$I_i(k)$ is measured motor current for joint i . $\phi_i(k)$ is a vector of measurable features. θ_i contains unknown coefficients. $e_i(k)$ is the part the model cannot explain.

3. Model A: No Acceleration

$$\phi_A = [1, \dot{q}_i, \tanh(\dot{q}_i / v_s), \sin(q_i), \cos(q_i)]$$

$$\hat{I}_i = \theta_0 + \theta_1 \dot{q}_i + \theta_2 \tanh(\dot{q}_i / v_s) + \theta_3 \sin(q_i) + \theta_4 \cos(q_i)$$

Constant term 1. The coefficient θ_0 absorbs baseline current offset and effects that look approximately constant over the sampled operating region.

Velocity $\dot{q}$. A simple viscous-friction model is $\tau_v = b \dot{q}$. The faster the joint moves, the larger this approximately linear component becomes. The fitted coefficient absorbs the unknown proportionality between joint velocity and measured current.

$\tanh(\dot{q} / v_s)$. Coulomb friction changes sign with motion direction. $\text{sign}(\dot{q})$ is discontinuous at zero, so $\tanh$ provides a smooth approximation that tends toward +1 for positive motion and -1 for negative motion. The scale v_s controls how quickly the transition occurs near zero velocity.

$\sin(q)$ and $\cos(q)$. Gravity torques are trigonometric functions of joint angle. Using both terms can represent a sinusoid with an unknown phase offset:

$$A \sin(q + \delta) = A \cos(\delta) \sin(q) + A \sin(\delta) \cos(q)$$

This is useful because the robot zero-angle convention does not have to coincide with the idealized physical gravity zero. The two coefficients can learn both magnitude and phase of a simple gravity component.

4. Model B: Add Acceleration

$$\phi_B = [1, q''_i, q'_i, \tanh(q'_i/v_s), \sin(q_i), \cos(q_i)]$$

Acceleration enters because rotational inertia follows the same idea as Newton's law:

$$\tau_{\text{inertia}} \approx J q''$$

So acceleration can improve current prediction during transients. But numerical differentiation amplifies measurement noise. With non-uniform timestamps, acceleration should use the measured interval

$$\Delta t_k = t_k - t_{(k-1)}$$

$$q''_k \approx [q'_f(k) - q'_f(k-1)] / \Delta t_k$$

A time-aware first-order velocity filter can be written as

$$\alpha_k = 1 - \exp(-\Delta t_k / \tau_f)$$

$$q'_f(k) = q'_f(k-1) + \alpha_k [q'(k) - q'_f(k-1)]$$

The point of Model B is not to assume acceleration belongs in the final model. It is to test that hypothesis. Keep q'' only if held-out prediction improves enough to justify the added noise and parameter sensitivity.

5. Offline Least Squares: How the Parameters Are Learned

For N samples from one joint, stack the measured currents into a vector y and the feature rows into the design matrix Φ :

$$y = \Phi \theta + \varepsilon$$

The least-squares estimate minimizes total squared prediction error:

$$\theta = \arg \min_{\theta} || y - \Phi \theta ||^2_2$$

The familiar closed form is $(\Phi^T \Phi)^{-1} \Phi^T y$, but the implementation should not explicitly invert $\Phi^T \Phi$. Use a numerically stable QR/SVD-backed least-squares solver such as `numpy.linalg.lstsq`.

```
theta_hat, residuals, rank, singular_values = np.linalg.lstsq(Phi_train, y_train, rcond=None)
```

The regression condition number is also important. If columns in Φ are nearly redundant, parameter estimates can become unstable even when prediction error looks acceptable. That is why the roadmap asks for both held-out error metrics and a condition-number check.

6. Residual: The Signal That Powers the Digital Twin

$$r_k = I_k - \hat{I}_k$$

During well-modeled normal operation, r_k should fluctuate around a small value. If an external interaction forces the motor to produce extra torque, measured current can increase while the normal-current model has not changed. Then the residual carries part of that unexplained load.

$$I_{\text{measured}} = I_{\text{normal}} + I_{\text{external}} \Rightarrow r \approx I_{\text{external}}$$

But a nonzero residual does not automatically mean collision. It can also come from thermal drift, payload, sensor noise, acceleration error, or an imperfect gravity/friction model. This is why adaptation and context logic are required.

7. Fixed-Forgetting-Factor RLS

Offline least squares fits one model from a batch. Recursive Least Squares updates the model sample by sample. With forgetting factor λ , the estimator approximately minimizes

$$J_k(\theta) = \sum_{j=1..k} \lambda^{(k-j)} [I_j - \phi_j^T \theta]^2$$

The main recursive quantities are the a-priori residual, gain, parameter update, and covariance update:

$$e_k^- = I_k - \phi_k^T \theta_{(k-1)}$$

$$K_k = P_{(k-1)} \phi_k / [\lambda + \phi_k^T P_{(k-1)} \phi_k]$$

$$\theta_k = \theta_{(k-1)} + K_k e_k^-$$

$$P_k = (1/\lambda) [P_{(k-1)} - K_k \phi_k^T P_{(k-1)}]$$

The current repository baseline uses fixed $\lambda = 0.98$. A useful rough effective-memory estimate is $1/(1-\lambda)$, which gives about 50 samples. The half-life in samples is $\ln(0.5)/\ln(\lambda)$, about 34.3 samples for $\lambda = 0.98$.

Because your logger is not uniformly sampled, a fixed per-sample λ does not represent an exactly fixed physical time horizon. Later, a time-aware forgetting formulation such as $\lambda_k = \exp(-\Delta t_k/\tau)$ may be worth evaluating, but it should not be inserted into the live estimator before the offline model and VFF design are frozen.

8. Why Variable Forgetting Factor RLS Is Needed

A large λ gives stable parameters but slow adaptation. A smaller λ adapts faster but can chase noise and transient disturbances. VFF-RLS changes λ according to evidence that the normal model is drifting. The roadmap calls for four ingredients: normalized a-priori residual, a robust slow residual-energy statistic, a $\lambda_{\min}/\lambda_{\max}$ law, and smoothing.

$$z_k = e_k^- / (\hat{\sigma}_e + \varepsilon)$$

$$S_k = (1-\alpha) S_{(k-1)} + \alpha \rho(z_k^2)$$

$$\lambda_k = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) g(S_k)$$

Here z_k puts residuals on a comparable scale, S_k measures persistent residual energy, ρ limits the effect of extreme outliers, and $g(S)$ should decrease as residual energy grows. In normal stable operation, λ should stay near $\lambda_{\max}$. During slow genuine drift, λ can decrease so the model adapts faster.

9. The Critical Context Gate: Do Not Learn a Collision

A naive adaptive estimator sees a large collision residual and concludes that the model is wrong, so it adapts faster. That can make the estimator absorb the collision current into its notion of normal behavior. The context-aware design prevents this by freezing parameter adaptation during collision/contact candidates or unresolved payload events.

$a_k = 1$ if adaptation is allowed; $a_k = 0$ if adaptation is frozen

$$\theta_k = \theta_{(k-1)} + a_k K_k e_k$$

This is the central conceptual distinction of the proposed context-aware Digital Twin: slow environmental/model drift may be learned; transient interaction events must first be protected from being learned as normal.

10. CUSUM: Accumulating Evidence

A single residual threshold can be sensitive to noise. CUSUM accumulates repeated deviations:

$$S^+_k = \max(0, S^+_{(k-1)} + z_k - \kappa)$$

$$S^-_k = \max(0, S^-_{(k-1)} - z_k - \kappa)$$

Trigger if $S^+_k > h$ or $S^-_k > h$

κ is a reference allowance that ignores small deviations; h is the accumulated-evidence threshold. A single small noisy event can decay back toward zero, while a persistent shift keeps accumulating. This makes CUSUM useful for sustained, modest departures from the learned baseline.

11. Why Payload Inference Should Use Stationary + Grasp Context

When the robot is approximately stationary, $\dot{q} \approx 0$ and $\ddot{q} \approx 0$, so velocity and inertial terms are minimized. A residual shift after a known grasp is therefore easier to associate with additional gravity load. A simple calibration model at a fixed pose is

$$\tilde{r}_i = b_i + s_i m$$

$$\hat{m} = (\tilde{r}_i - b_i) / s_i$$

The planned 0 / 100 / 250 / 500 g calibration trials can be used to estimate sensitivity, bias, uncertainty, and whether one joint or a weighted multi-joint estimate is more reliable.

12. End-to-End Mathematical Pipeline

1. Telemetry	$q, \dot{q}$, optional $\ddot{q}$, measured current I , real timestamps
2. Normal-current model	$\hat{I} = \phi^T \theta$
3. Residual	$r = I - \hat{I}$
4. Statistical processing	normalize residual, estimate slow residual energy, CUSUM / persistence logic
5. Context	motion state, stationary state, grasp context, contact candidate, timing/hysteresis

6. Action

allow adaptation for drift; freeze for contact/collision; infer payload only in valid grasp context

The immediate research gate

Before VFF, thresholds, or state-machine tuning, produce a per-joint table comparing Model A and Model B on identical held-out runs. At minimum report train RMSE, held-out RMSE, MAE, residual mean/std, and the regression condition number. Then inspect residual plots rather than choosing only from a single scalar metric.

Joint	Model	Train RMSE	Held-out RMSE	MAE	Residual mean	Residual std	cond(Φ)
J1	A	...	...	...	...	...	...
J1	B	...	...	...	...	...	...
J2	A	...	...	...	...	...	...

13. Lab Readiness Checklist

- Raw telemetry is logged before any RLS, CUSUM, filtering, or anomaly transformation.
- Timestamp is monotonic; actual Δt distribution is inspected and saved.
- Units are explicit. Preserve original units; create separate modeling columns in rad, rad/s, rad/s², and A.
- Each row can be traced to experiment_id, trajectory_id, and run_id.
- First identification data use payload_g = 0 and contact_label = NORMAL.
- The run plan includes repeated trajectories so entire runs can be held out.
- Current saturation/clipping and missing values are checked before long data collection.
- The existing live FF-RLS/CUSUM may display results, but it must not alter raw telemetry or become the regression target.
- No VFF law, new state-machine guards, or final thresholds are invented during this logger change.

14. Ready-to-Paste Prompt for GitHub Copilot Agent

Copy the prompt below into Copilot Agent while it has the repository open. It is intentionally narrow: collect identification-quality data without redesigning the research method.

You are working in this repository:
dungnguyen15213/6D0F-Rebel-Cobot-Control

GOAL

Modify the repository only as much as necessary to make it ready to collect an identification-quality telemetry dataset for offline motor-current modeling of the 6-DOF igus ReBeL.

IMPORTANT RESEARCH CONSTRAINTS

1. Do NOT change the existing live RLS mathematics, forgetting factor, CUSUM equations, payload estimator logic, or state-machine logic as part of this task.
2. Do NOT implement VFF-RLS yet.
3. Do NOT invent new collision/contact thresholds, VFF laws, regressor physics, or calibration constants.
4. Do NOT use the adaptive RLS estimate/output as the target for offline system identification. The target must be the measured/raw motor current from CRI STATUS telemetry.
5. Existing FF-RLS/CUSUM may continue running for display, but they must not modify, replace, filter, or overwrite raw telemetry fields.
6. Preserve backward compatibility with the current GUI and normal robot operation as much as practical.
7. Keep the changes minimal, modular, and easy to revert.

FIRST: INSPECT BEFORE EDITING

- Inspect the repository tree and identify exactly where CRI STATUS telemetry is received, where robot_diagnostics.csv (or equivalent diagnostics logging) is written, how timestamps are generated, and where path/trajectory execution metadata is available.
- Identify the current units for q, velocity, acceleration, and current.
- Identify whether the logged position/current values are truly raw STATUS values or already transformed/filtered.
- Summarize your findings before making edits.

IMPLEMENT A DEDICATED IDENTIFICATION LOGGER

Create a dedicated experiment logger/output for system-identification runs rather than relying on an arbitrary continuously reused diagnostics CSV.

Required raw/sample columns:

- timestamp
- q1 ... q6
- qdot1 ... qdot6
- current1 ... current6

Candidate/optional columns if already available without changing control behavior:

- qddot1 ... qddot6

Required run metadata on every sample or in a losslessly joinable run metadata file:

- experiment_id
- trajectory_id
- run_id
- commanded_speed
- payload_g
- contact_label
- grasp_context
- thermal_condition (or equivalent cold/warm field)

For the first identification experiments, defaults should be easy to set to:

- payload_g = 0
- contact_label = NORMAL
- grasp_context = NONE / FALSE as appropriate

TIMESTAMP REQUIREMENTS

- Use the best monotonic/sample timestamp already available in the application.
- Do NOT assume a fixed sample period.
- Preserve enough precision so offline code can calculate:
 $dt[k] = timestamp[k] - timestamp[k-1]$
- Do not create qddot by assuming a constant dt.

RAW DATA REQUIREMENTS

- Preserve raw telemetry values exactly in dedicated raw columns.
- Never overwrite raw q, qdot, current, or timestamp columns with filtered/processed values.
- If you add converted or processed signals, put them in additional explicitly named columns.
- Keep units explicit in code/comments/documentation. If the live logger currently uses degrees, deg/s, deg/s^2, and mA, keep those raw units; offline analysis will create rad/rad/s/rad/s^2/A columns.

FILE / RUN BEHAVIOR

- Prefer one clearly named CSV per identification run, OR one experiment file with robust run_id separation. Choose the option that best fits the current architecture with the smallest safe change.
- Do not silently overwrite earlier experiment data.
- Include a clear file naming convention with experiment/run identity and timestamp when appropriate.
- Flush/close safely at the end of a run so interrupted experiments do not lose all data.

DATA-QUALITY CHECK / DRY-RUN SUPPORT

Add either a small utility script or a clearly documented command/function that can inspect a newly recorded identification CSV and report at least:

- row count
- timestamp monotonicity
- duplicate timestamps
- min/median/max dt and useful percentiles
- missing/NaN values by column
- current min/max by joint
- likely current clipping/saturation if an explicit known hardware limit exists in the repo; otherwise report extrema without inventing a limit
- units/schema summary
- run_id / trajectory_id values found

Do not add offline Model A / Model B fitting into the live robot-control path. A separate offline analysis script/notebook can be added only if it is cleanly separated from live execution.

TESTING

Add or update lightweight tests where practical for:

- logger schema
- run metadata propagation
- timestamp handling
- file creation / non-overwrite behavior
- data-quality utility on a small synthetic CSV

Do not require physical robot hardware for these unit tests.

DOCUMENTATION

Add a short IDENTIFICATION_DATA_COLLECTION.md (or update an appropriate existing guide) that explains:

1. how to start an identification run;
2. how to set experiment_id, trajectory_id, run_id, commanded_speed, payload_g, contact_label, grasp_context, and cold/warm condition;
3. where the CSV is saved;
4. exact column names and units;
5. how to stop/close the run safely;
6. how to run the data-quality check;
7. the intended first experiment condition: no payload, no intentional contact, no collision.

DO NOT DO YET

- no VFF-RLS implementation
- no context-state-machine redesign
- no collision/soft-contact threshold tuning
- no payload calibration changes
- no final acceleration reconstruction/filter design in the live code
- no arbitrary cross-axis gravity sine terms
- no GUI redesign unless a tiny field/button is strictly necessary to supply run metadata

ACCEPTANCE CRITERIA

The task is complete when:

- A. I can run the robot normally and start an identification recording without breaking the existing application.
- B. The output contains timestamp + ql.q6 + qdot1..qdot6 + current1..6 and traceable run metadata.
- C. Raw telemetry is not overwritten by processed analytics.
- D. I can repeat the same trajectory as separate run IDs.
- E. A dry-run quality check shows the actual dt distribution and flags missing/duplicate data.
- F. Existing RLS/CUSUM behavior remains mathematically unchanged.
- G. Tests pass (or you clearly explain any pre-existing failures unrelated to your changes).

BEFORE FINISHING

- Show me the files changed.
- Explain the purpose of each change.
- Show the final CSV schema with units.
- Show exactly how I launch/start one identification recording.
- Show exactly how I set metadata for a run.
- Show exactly how I run the dry-run data-quality check.
- Report tests executed and their results.
- Call out any uncertainty about what the CRI/API exposes as true raw velocity/current/timestamp instead of guessing.
- Do not proceed to VFF-RLS or Model A/Model B fitting unless I explicitly ask in a later task.

15. What to Bring Back After Copilot Finishes

- The Copilot summary of files changed and why.
- The exact final CSV schema and units.
- One short dry-run CSV (30-120 seconds is enough initially).
- The dry-run quality-check output, especially Δt statistics and current ranges.
- The command or GUI procedure used to start/stop recording and set run metadata.
- Any uncertainty Copilot found about whether velocity/current/timestamps are raw or derived.

With those artifacts, the next step is to inspect the data quality and build the offline preprocessing + Model A notebook. Model B should be added only after Model A is working and the acceleration signal has been checked.